EDGE GATEKEEPER · FUTURE ROADMAP

From Prototype to Production — and Beyond

Where the Gatekeeper sits in the full ambient-intelligence pipeline, and the sequential plan — with requirements — to align it and reach production

P0 · Foundation
P1 · Pipeline Interface
P2 · Context
P3 · Feedback
P4 · Real Data
P5 · Semantic
P6 · On-Device
P7 · Production

Generated 2026-07-20 · edge-gatekeeper project

1. Where the Gatekeeper sits in the full system

The Gatekeeper is one box in a larger, mostly-unbuilt pipeline. Everything else in the diagram below — the wearable, its audio transport, the speech-to-text engine, and the downstream "deeper intelligence" — is explicitly out of scope for this project and does not exist anywhere in this workspace. Before planning enhancements, it's worth being precise about that boundary.

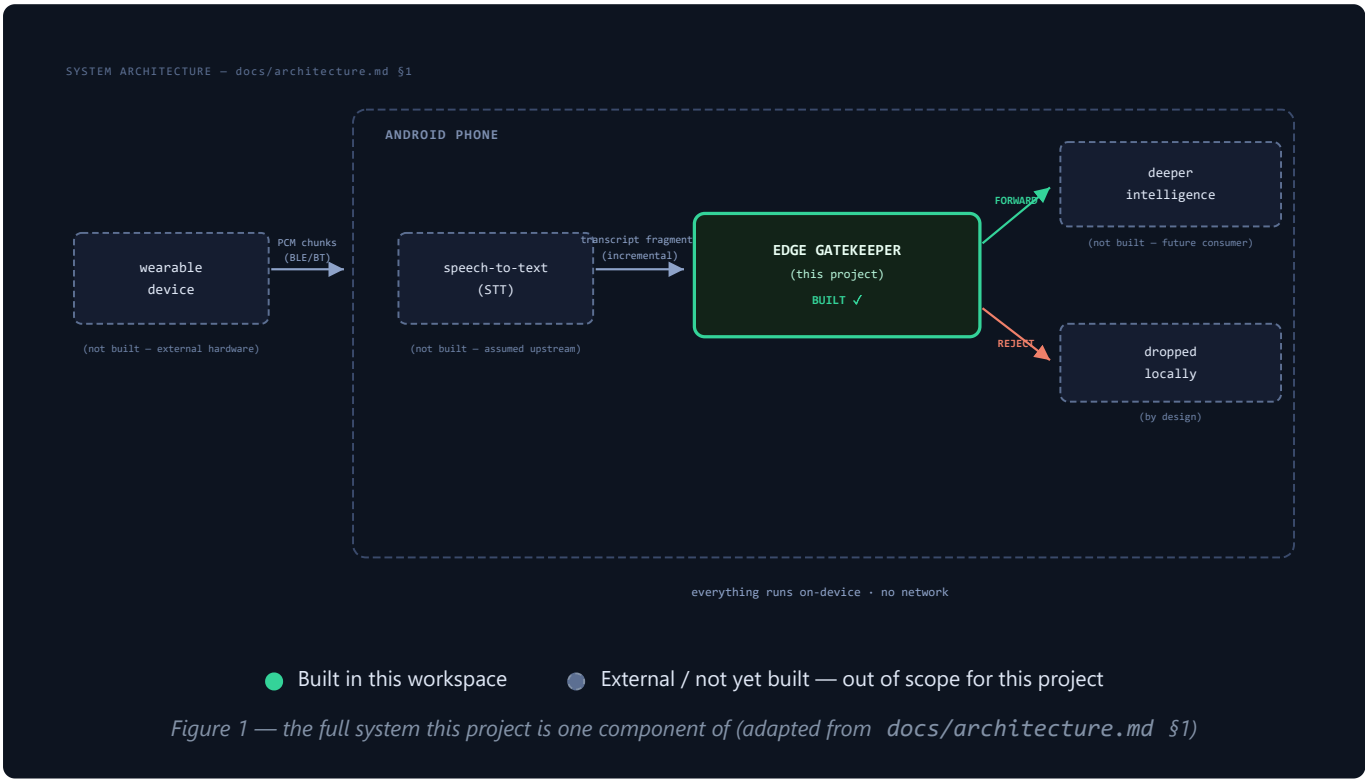

Figure 1 — the full system this project is one component of (adapted from `docs/architecture.md` §1)

Current position

Component	Status	Notes
Wearable device (audio capture)	Not built	External hardware, out of scope; assumed to exist and stream audio reliably
PCM chunks over BLE/BT	Not built	Out of scope; assumed low-latency and reliable
Speech-to-text (STT)	Not built	Out of scope; this project's synthetic training data mimics its *output style* (lowercase, no punctuation, fillers) — not its internals
EDGE GATEKEEPER	**Built — this project**	TF-IDF + LogisticRegression engine, FastAPI service, demo console; recently enhanced (scenario suite 18/22 → 20/22, true-drop rate 0.14% → 0.00% — see `Enhancement_Report.pdf`)
Deeper intelligence	Not built	Out of scope; today a `FORWARD` decision only reaches the demo frontend, not a real consumer

> **WHAT THIS MEANS FOR THE ROADMAP**
>
> The wearable, its transport, the STT engine, and the downstream agent are each somebody else's component to build, and stay out of scope here (per the original assignment brief). What *is* this project's responsibility is making sure the Gatekeeper is a well-behaved neighbor to all four of them — that it can absorb what a real STT engine actually outputs, and that it hands off decisions through an interface a real downstream system could actually be built against. That is exactly what Phase 1 below addresses, and it's why it comes before every other enhancement in this plan: every later phase (context-awareness, real data, the semantic re-scorer, the Kotlin port) is easier to build correctly once these two seams are solid.

2. How to read this roadmap

This document lays out eight sequential phases — each one buildable on top of the last, starting from the model retrain already committed to this repository and ending at a genuinely production-ready on-device product that fits cleanly into Figure 1. Every phase lists: what changes, why it's next, and exactly what you'd need (data, engineering time, infrastructure, skills) to do it. A final note covers what this project would look like with no resource constraints at all.

Each phase carries badges:

- **LOW EFFORT** / **MEDIUM EFFORT** / **HIGH EFFORT** — engineering effort relative to the others here, not absolute difficulty.
- **TIMEFRAME** — a rough calendar estimate for one focused engineer (or small team where noted).
- **ALIGNMENT** — marks the phase directly driven by Figure 1's pipeline boundary.
- **MILESTONE** — marks the phase that represents "production-ready."

3. Roadmap at a glance

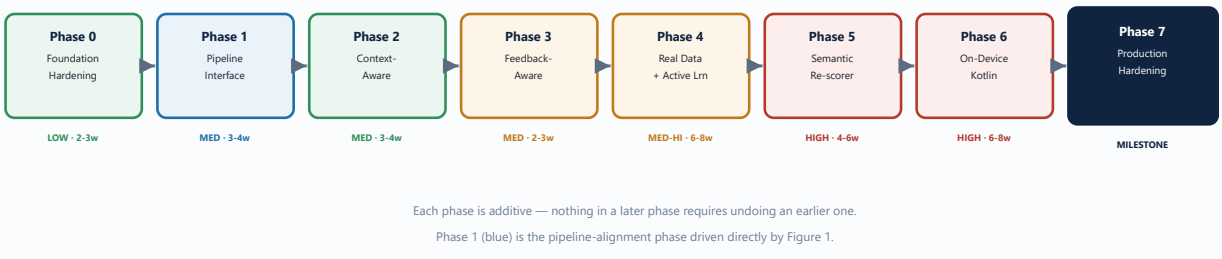

Figure 2 — the eight-phase roadmap, effort and rough timeframe per phase

4. Phase 0 — Foundation Hardening

Make the current classifier's fixes solid before building on them

LOW EFFORT 2-3 WEEKS

This phase closes out loose ends from the classifier enhancement already committed: the disclosed "vague future risk" trade-off, and the lack of any automated guard against future regressions.

- Properly resolve the disclosed regression on referent-free risk phrasing ("that might be a problem later") — likely needs a handful of real or carefully-designed synthetic examples distinct enough from the "task"/"decision" augmentation to not cross-contaminate them again.
- Expand the hand-written scenario suite beyond 22 turns — more multi-speaker conversations, more edge cases per category, so future retrains have a bigger safety net.
- Add a **regression gate**: a script that fails if a retrain drops the scenario-suite score or raises the true-drop rate above the last committed baseline — turns the manual verification done for this fix into something automatic.

Data

- A few dozen new synthetic phrases
- 5-10 new hand-written scenarios

Engineering

- Notebook/data science time only
- A small Python regression-check script

Infra

- None new — reuses `backend/simulator.py`

Skills

- Same skillset as the current codebase (pandas, sklearn)

5. Phase 1 — Pipeline Interface Hardening

Harden the two seams where the Gatekeeper meets the rest of Figure 1

ALIGNMENT | MEDIUM EFFORT | 3-4 WEEKS

Right now the Gatekeeper is trained and evaluated in isolation: its input is synthetic ASR-style text and its output goes to a demo frontend. This phase is the direct response to Figure 1 — it doesn't change what the Gatekeeper *decides*, it changes what it can safely *receive* and what it reliably *hands off*, so that a real STT engine and a real downstream agent could actually be plugged in on either side without surprises.

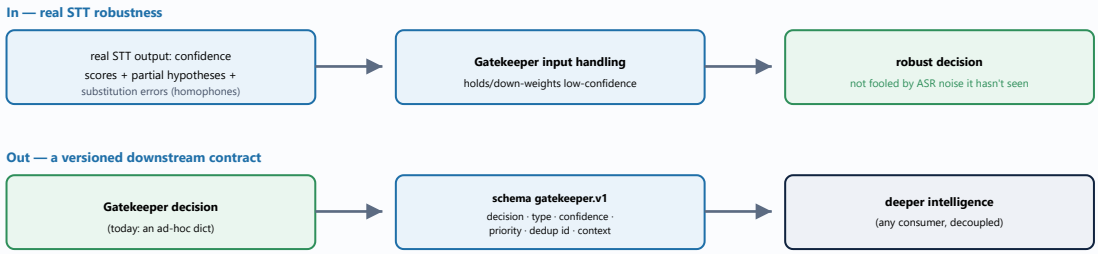

Figure 3 — hardening both seams from Figure 1: realistic ASR input on the way in, a versioned contract on the way out

- **Input:** extend the ASR noise model beyond lowercasing/fillers/word-repetition to include homophone/substitution errors ("their/there", "for/four", "won/one") and streaming partial hypotheses that get revised as more audio arrives — today the engine assumes every fragment it receives is final.
- Accept an optional per-fragment confidence/ `is_final` field; hold or down-weight low-confidence or non-final fragments instead of evaluating them as if certain.
- **Output:** replace the ad-hoc `GateResult` dict with a versioned, documented schema (`gatekeeper.v1`) — decision, moment type, confidence, a derived priority tier, dedup group id, context window, timestamp — that a downstream "deeper intelligence" consumer could be built against independently, without needing to track this project's internals.
- Derive a priority tier from decision + moment type (e.g. risk/decision → HIGH, task/info/question → NORMAL, UNCERTAIN → LOW) so downstream doesn't have to re-derive it.
- Version the API (`/api/v1/evaluate`) so the Gatekeeper keeps evolving without breaking whatever gets built against it later.

Data

- Realistic STT error examples (confidence scores, substitutions, revised hypotheses) — synthetic to start, same template+noise philosophy as today

Engineering

- Schema design (Pydantic) + API versioning
- `GatekeeperEngine` change to accept confidence/ `is_final`
- Priority-tier derivation logic

Infra

Skills

- None new

- API/contract design
- Some ASR domain familiarity, to model realistic error patterns

6. Phase 2 — Context-Aware Classification

Let the model use the previous turn — done right this time

MEDIUM EFFORT 3-4 WEEKS

A naive version of this (concatenating previous + current text into one input) was tried during the original build and reverted — it contaminated short ordinary lines with the previous line's vocabulary. The correct version keeps the current-utterance signal clean and adds context as a *separate* feature, not a blended one.

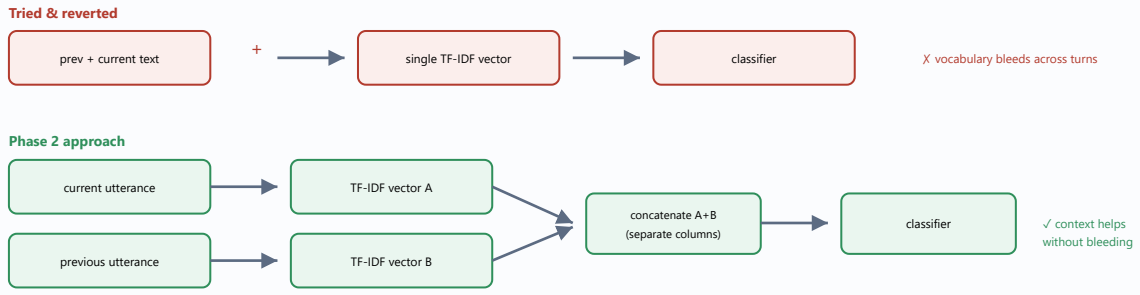

Figure 4 — why the previous attempt failed, and the separate-feature-columns approach for Phase 2

Data

- Multi-turn synthetic conversations (not just isolated utterances)
- Scenario suite turns that specifically depend on the prior line

Engineering

- Feature-union changes in `notebooks/02`
- `backend/gatekeeper.py` already tracks a context ring buffer — reuse it as the feature source

Infra

- None new

Skills

- Feature engineering, careful ablation testing (to avoid repeating the original contamination bug)

7. Phase 3 — Feedback-Aware Session State

Let downstream tell the Gatekeeper when a moment has been handled

MEDIUM EFFORT | 2-3 WEEKS

Today, duplicate suppression only knows about moments the Gatekeeper itself forwarded — it has no idea if downstream ever actually resolved them. This phase adds a lightweight, fully local feedback channel, built on top of Phase 1's versioned contract, so a resolved moment ("stove's off now, thanks") doesn't get re-forwarded on every restatement, and starts collecting the implicit signal (was a forward acted on or dismissed?) that Phase 7's adaptive thresholds will eventually need.

- New endpoint, e.g. `POST /api/v1/feedback`, marking a previously-forwarded moment (by its dedup group id from Phase 1) as handled/dismissed.
- Extend `GatekeeperEngine`'s duplicate memory with a "resolved" flag per entry.
- Start logging (locally, not centrally) forward-vs-dismissed counts per session as raw material for Phase 7.

Data

- None new — this is a logic/schema change

Engineering

- New Pydantic schema + FastAPI route
- `GatekeeperEngine` state extension

Infra

- None new (still local, no server-side storage)

Skills

- Same backend stack already in use (FastAPI, Pydantic)

8. Phase 4 — Real-Data Pilot & Active Learning Loop

Graduate from synthetic data to real, consented conversation data

MEDIUM-HIGH EFFORT **6-8 WEEKS, THEN ONGOING**

This is the first phase that needs people outside engineering: consented pilot users and someone doing annotation. The existing synthetic-data model is used to bootstrap the pool of candidates worth labelling.

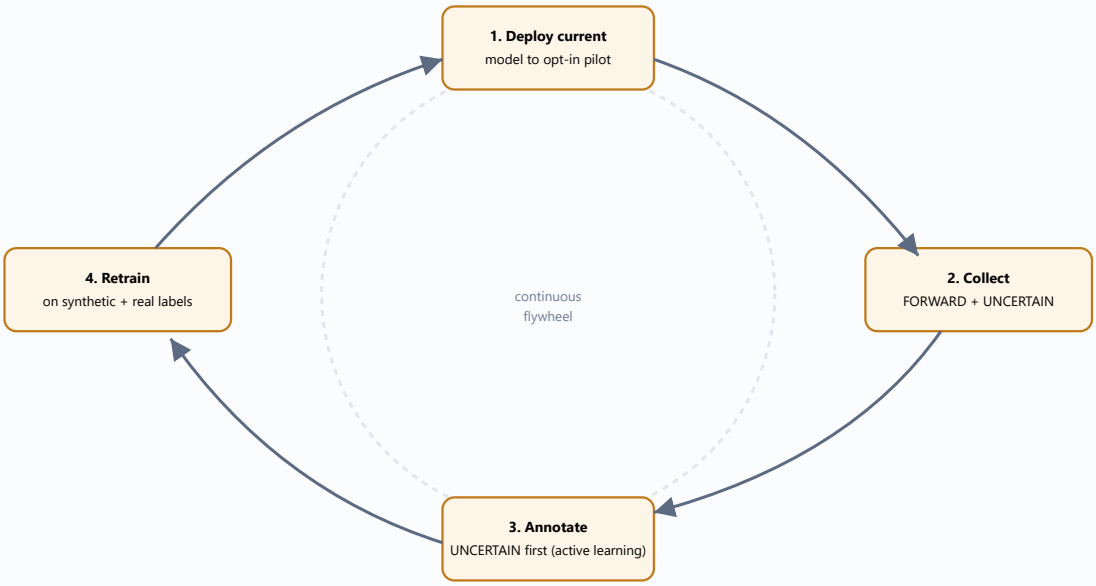

Figure 5 — the active-learning flywheel: label the model's own uncertainty band first, for the cheapest accuracy gain per label

- Opt-in pilot cohort with clear, revocable consent.
- Lightweight internal annotation tool: one fragment + 3 lines of prior context, two questions ("should this reach the assistant?" and "which moment type?") — matches the process already designed in `docs/data_strategy.md` §5.
- PII scrubbing before any storage; fragments only, never full-conversation export.
- Track inter-annotator agreement — low-agreement items become `UNCERTAIN` training targets rather than being forced into a class.

Data

- Consented pilot users (even 10-20 is a meaningful start)
- PII scrubbing pipeline

Engineering

- Small internal annotation web app
- Active-learning item-selection script

Infra

- Minimal secure storage for consented fragments only

People

- 1-2 part-time annotators

- Privacy/legal sign-off on the consent flow

9. Phase 5 — Semantic Second-Stage Re-scorer

Add paraphrase understanding, without leaving the 25 MB budget

HIGH EFFORT **4-6 WEEKS**

The cheap linear model handles the confident majority of fragments in ~1-2 ms. This phase adds a small quantised encoder that *only* runs on the minority landing in the `UNCERTAIN` band — the place a semantic model earns its keep, without paying its cost on every single fragment.

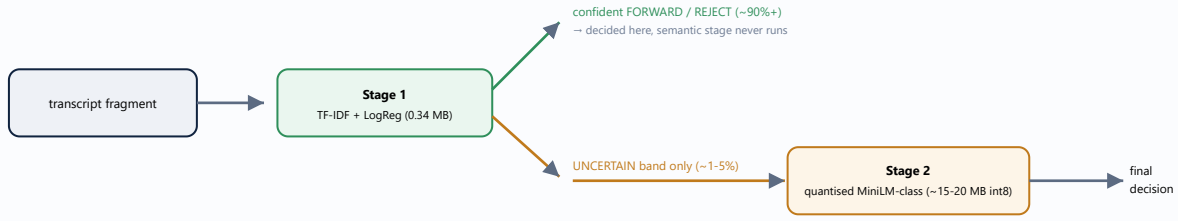

Figure 6 — two-stage cascade: the semantic model only runs on the grey band, keeping total assets under 25 MB

- Decide the model-provenance question up front: train a small encoder from scratch (keeps the original "no external pretrained models" constraint) vs. adopt a permissively-licensed small pretrained encoder (faster, but changes the submission's compliance story).
- Quantise to int8, export via ONNX, and budget-check: current assets (0.34 MB) + encoder (~15-20 MB) must stay under 25 MB.
- Re-run the full evaluation suite with the cascade active — the win to prove is fewer paraphrase-blind misses in the `UNCERTAIN` band specifically, not a change to the Stage-1 numbers.

Data

- Reuses existing labelled data; may need paraphrase-focused eval examples

Engineering

- ONNX export/quantisation tooling
- Cascade logic in `backend/gatekeeper.py`

Infra

- ONNX Runtime (CPU) for serving; a GPU is only needed to train/distill, not to run

Skills

- Model compression/quantisation experience

10. Phase 6 — On-Device Android/Kotlin Port

Move off the FastAPI prototype and onto the actual target device from Figure 1

HIGH EFFORT | 6-8 WEEKS

`docs/architecture.md` §5 already scopes this: the sparse linear model is two vocabulary hash maps, a coefficient matrix, and a softmax — reimplementable directly in Kotlin in roughly 200 lines, with no ML runtime dependency at all. The regex back-channel filter, threshold gate, duplicate cosine check and ring buffers port over just as directly.

- Export vectorizer + classifier weights to a flat binary/JSON (<1 MB).
- Reimplement transform → dot-product → softmax in Kotlin; port the deterministic layers unchanged.
- If Phase 5 shipped, embed ONNX Runtime Mobile (~5 MB APK overhead, outside the 25 MB model budget) for the semantic stage only.
- Battery/latency benchmarking on a real mid-range device, not just a laptop CPU proxy — and benchmark *relative to* the STT engine's own always-on cost from Figure 1, since that already runs continuously on the same phone; the Gatekeeper is not the primary battery draw in this pipeline.

Data

- None new — this is a faithful port, not a retrain

Engineering

- Android/Kotlin engineer
- Cross-checking harness comparing Kotlin output vs. the Python engine bit-for-bit on the scenario suite

Infra

- A small device lab spanning a few chipsets/RAM tiers
- Battery profiling tooling (Android Studio Profiler / Battery Historian)

Skills

- Kotlin, Android platform APIs, on-device performance profiling

11. Phase 7 — Production Hardening & Scale-Out (the production-ready milestone)

Everything a real, shipping product needs beyond a working model

HIGH EFFORT | 2-3 MONTHS | MILESTONE

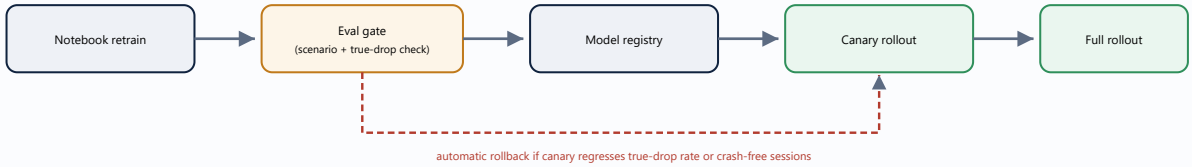

Figure 7 — the retrain-to-rollout pipeline that turns Phase 0's regression gate into a real release process

- **Multi-language expansion** — Hinglish first (the char n-gram design was chosen partly for this), then further languages, each needing its own template/vocabulary pass and evaluation.
- **Per-user adaptive thresholds** — graduate Phase 3's raw feedback logs into an actual learned personalization layer (implicit feedback nudges the forward threshold per user).
- **Observability** — aggregated, privacy-safe telemetry on decision latency, crash rates, and forward/reject/uncertain ratios in the field (never raw transcript content).
- **CI/CD** — the pipeline in Figure 7, wired to real infrastructure, with staged canary rollout and automatic rollback.
- **Privacy & compliance** — formal Data Protection Impact Assessment given this is an always-on ambient-audio-adjacent product; on-device encryption of any cached fragments; a clear data retention policy.
- **A/B testing infrastructure** for threshold and model changes before full rollout.

Data

- Aggregated field telemetry (opt-in, privacy-preserving)

Engineering

- SRE / mobile-infra engineer
- CI infra (e.g. GitHub Actions) wired to the notebook + simulator gate

Infra

- Model registry, canary rollout tooling, telemetry backend

People

- Privacy/legal counsel for the DPIA
- On-call rotation once this is a real release

12. Production readiness checklist

Area	Done by which phase	Status if all phases complete
Core model quality (binary gate)	Phase 0 (already underway)	Automated regression gate in place
Pipeline interface contract	Phase 1	Real-ASR robustness handled; versioned I/O schema in place
Multi-turn context	Phase 2	Context used safely, no contamination
Feedback loop	Phase 3 → 7	From logging-only to full personalization
Real-world data	Phase 4	Continuous active-learning pipeline
Paraphrase robustness	Phase 5	Cascade re-scorer live, still <25 MB
Runs on target hardware	Phase 6	Native Kotlin, benchmarked on real devices
Release safety net	Phase 7	CI gate + canary + rollback
Privacy/compliance sign-off	Phase 7	DPIA complete, retention policy live
Multi-language support	Phase 7	Hinglish shipped, others planned

13. Note — the greatest possible version of this project, with unlimited resources

> **IF BUDGET, HEADCOUNT AND TIME WERE NOT CONSTRAINTS**
>
> Every phase above is scoped to be achievable by a small team on a realistic timeline, aligning the Gatekeeper to the pipeline in Figure 1 as it exists today. With no resource constraints at all, the ambition changes: instead of just aligning to Figure 1, you'd rebuild every box in it to a much higher standard, and add the pieces the diagram doesn't even show yet. Here is the structure that would get there, and what it would actually take.

The ideal structure

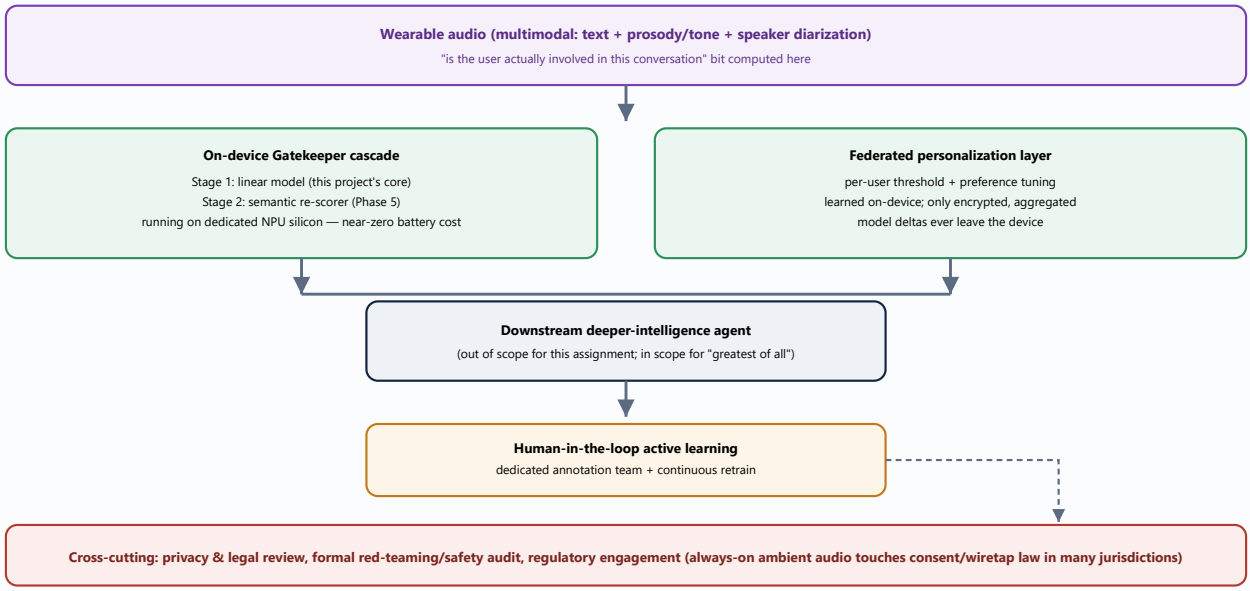

Figure 8 — the unlimited-resources architecture: multimodal input, an on-device cascade with federated personalization, the downstream agent this project explicitly stops short of, a continuous human-in-the-loop learning system, and privacy/legal review wrapping the whole stack

What it would take

Team

- ML researchers (multimodal + on-device efficiency)
- Mobile/embedded engineers (Android, NPU/silicon integration)
- Privacy & legal counsel (dedicated, not advisory)
- UX researchers (trust, consent, notification design)
- A standing annotation team
- SRE/on-call for a real always-on service

Data & learning infrastructure

- Federated learning infrastructure (e.g. secure aggregation, differential privacy) so personalization never centralizes raw audio/text
- A continuously-running active-learning loop, not a one-time pilot
- Multilingual data collection across target markets

Hardware

- NPU/custom-silicon partnership for always-on inference at near-zero battery cost
- A large, diverse on-device benchmarking lab

Trust & safety

- Formal adversarial red-teaming — this listens to everything around the user, so the stakes for getting it wrong are unusually high
- Regulatory engagement per market (consent/wiretapping law varies widely for always-on audio)
- A published, auditable privacy policy and data retention model

The throughline across every phase in this document, resourced or not, is the same one this project already follows: measure everything, disclose trade-offs honestly, and never let a confident-sounding model make a decision it hasn't earned the right to make.